

CONTENTS

- Overview
- 1 What an Agentic Framework Actually Coordinates
- 2 Linear Workflows
- 3 Autonomous Multi-Agent Systems
- 4 Role-Based Multi-Agent Systems
- 5 Production Orchestration Systems
- 6 Rapid Prototyping Tools
- Glossary
- Check yourself
- Where to go next

The Five Types of Agentic AI Systems (And Which Framework Fits Each)

Learn to classify an agentic AI project by its coordination pattern so you can pick a framework instead of guessing from a list of GitHub stars.

For developers evaluating LangChain, CrewAI, AutoGen, or similar tools who need a mental model for which one actually fits their project · 27 July 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

BEFORE YOU START

- Basic familiarity with what a large language model (LLM) is and how it responds to prompts
- Some exposure to the idea of an 'AI agent' that can call tools or take actions rather than just answer questions

Overview

Agentic AI development has an ecosystem problem: LangChain, LangGraph, CrewAI, AutoGen, Semantic Kernel, LangFlow, and half a dozen others all present themselves as the right way to build an agent. They are not interchangeable. Each was designed around a different shape of coordination problem, and picking one before you know your system's shape is a common way projects end up rebuilt from scratch.

An agentic AI framework's job is to manage the coordination that appears once an LLM-based system does more than answer a single question. An agent that queries a database, runs a calculation, and drafts a report already has to plan its steps, call tools, hold state between them, and recover from failures — a chatbot's single call to a model does none of that. Frameworks exist to supply the reusable parts for that coordination: predefined architectures for common agent shapes, task and state management, integration and monitoring tooling, and communication protocols for when more than one agent is involved.

Because agent projects don't all need the same kind of coordination, they cluster into five recognizable patterns: linear workflows, autonomous multi-agent systems, role-based multi-agent systems, production orchestration, and rapid prototyping. Naming which pattern a project fits narrows the framework choice from a dozen options down to two or three, and turns "which framework is best" into the more answerable "what am I actually building."

KEY TAKEAWAYS

- ✓ An agentic framework's core job is coordinating planning, tool calls, and state across steps or agents — work a single LLM call never has to do.
- ✓ The useful question is 'what shape is my system?', not 'which framework is best?' — the five patterns map almost directly onto specific tools.
- ✓ Linear workflows (LangChain, LlamaIndex, LangGraph) suit tasks with a fixed, predictable sequence of steps and no need for agents to negotiate with each other.
- ✓ Autonomous multi-agent systems (AutoGen, CrewAI, BabyAGI) suit open-ended goals where agents work out their own division of labor.
- ✓ Role-based multi-agent systems (CrewAI, structured AutoGen, ChatDev) look like autonomous systems but confine each agent to a fixed role boundary, trading flexibility for predictability.
- ✓ Production orchestration frameworks (Microsoft's Agent Framework, LangGraph) add the deep API and database integration and the reliability a system needs once it leaves experimentation.
- ✓ Rapid prototyping tools (LangFlow, Flowise) trade architectural rigor for a drag-and-drop canvas that validates an idea before you invest in a real build.

01 What an Agentic Framework Actually Coordinates

1:24

An agentic framework exists because doing more than one LLM call — planning, tool use, iteration, and often inter-agent communication — needs reusable coordination machinery that a plain chatbot never requires.

Consider an agent built to analyze sales data. It has to go to a database and extract the relevant rows, run calculations over that data, generate a report from the results, and craft a response for whoever asked. Each of those is a distinct step with its own inputs and failure modes, and the agent has to move between them in the right order, carrying state forward as it goes. A chatbot that just answers a question does not have this problem — one prompt goes in, one completion comes out.

The complexity compounds once more than one agent is involved, or once the task branches based on what an earlier step returned. Someone has to decide how state is passed between steps, what happens when a tool call fails, and — if there are multiple agents — how they exchange information. Doing all of that by hand for every project is what an agentic framework is built to avoid.

Concretely, frameworks tend to supply four recurring pieces: predefined architectures for common agent shapes (so you are not designing the control flow from scratch), task and state management, integration and monitoring tooling, and communication protocols for when several agents need to talk to each other. Which of these four matter most, and how much structure you want around them, is what actually separates one framework from another.

KEY POINTS

- A chatbot makes one call and returns one answer; an agent plans, acts, observes results, and often iterates before it is done.
- Frameworks supply four recurring pieces: predefined architectures, task/state management, integration and monitoring tools, and communication protocols for multi-agent setups.
- The more coordination a system needs — more steps, more agents, more external systems — the more a framework's overhead is worth paying.

What a framework replaces

Without a framework, a developer hand-writes the orchestration a sales-report agent needs: calling the database, running calculations, generating a report, deciding whether to iterate, and sending the result. This is exactly the coordination logic that frameworks abstract into reusable components.

```
# The coordination you'd otherwise write and maintain by hand
def sales_report_agent(question):
    data = query_database(question)          # tool call
    analysis = run_calculations(data)         # tool call
    report = llm_generate_report(analysis)    # LLM call: decide what to include
    if needs_followup(report):                # iteration / branching
        report = refine_report(report, data)
    send_response(report)                    # tool call
    return report
```

02 Linear Workflows

3:30

When steps happen in a fixed, predictable sequence, a linear workflow gives full control and reliability without the overhead of agents negotiating with each other.

In a linear workflow, the steps and their order are decided at design time, not worked out by the system at runtime. A customer support agent is a typical example: a user asks a question, the agent searches a knowledge base for relevant material, crafts a response from what it finds, and — if the question stays unresolved — creates a support ticket. That is always the sequence; nothing about it needs to be negotiated between multiple agents each run.

This pattern is the right choice when reliability matters more than flexibility. Because the flow is fixed, it is also easier to test, debug, and reason about than a system where agents can take different paths each time. There is no need for multiple agents to collaborate to make it happen — a single agent, or a chain of tool calls, is enough.

LangChain is well suited to setups where multiple steps need to happen in a defined sequence. LlamaIndex specializes in the same kind of pipeline when it is heavy on data retrieval and indexing rather than general tool use. For more complex versions of a linear flow — ones with branching or loops but still a knowable structure — LangGraph, also from the LangChain team, extends the same idea to a graph instead of a strict list.

KEY POINTS

- Steps run in a known order decided at design time, not negotiated at runtime.
- No inter-agent negotiation is needed — a single agent or tool chain executes the sequence.
- Best when reliability and predictability matter more than flexibility.
- LangChain suits general multi-step pipelines; LlamaIndex suits retrieval- and indexing-heavy pipelines; LangGraph extends the pattern to branching, stateful flows.

Customer support pipeline

A support bot always runs the same three steps in order: find the relevant article, draft a reply from it, and escalate to a ticket only if the reply doesn't resolve the question. Written as a LangChain-style chain, each step's output feeds directly into the next.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

search_prompt = ChatPromptTemplate.from_template(
    "Find the most relevant knowledge-base article for: {question}"
)
reply_prompt = ChatPromptTemplate.from_template(
    "Using this article, answer the user's question.\n\nArticle: {article}\nQuestion: {question}"
)

find_article = search_prompt | llm | StrOutputParser()
craft_reply = reply_prompt | llm | StrOutputParser()

def handle_ticket(question):
    article = find_article.invoke({"question": question})
    reply = craft_reply.invoke({"article": article, "question": question})
    if reply_unresolved(reply):
        create_support_ticket(question, reply)
    return reply
```

03 Autonomous Multi-Agent Systems

4:54

Give the system a goal instead of a procedure, and multiple agents figure out the steps and division of labor between them — the pattern for open-ended problems.

In an autonomous agentic system, you typically hand the system a goal rather than a sequence of steps, and it is common for multiple agents to collaborate to reach it. An AI coding assistant is a good example: a planner agent breaks the task down, a coder agent writes the code, and a reviewer agent checks the result, makes recommendations, and helps with debugging. These agents talk to each other continuously, and the exact path they take to a working solution is not fixed in advance.

This pattern fits open-ended problems — ones where you cannot fully specify the steps up front because the right approach depends on what earlier steps discover. That is also its cost: because the agents work out their own plan, behavior is less predictable than a linear workflow, and debugging means inspecting a conversation between agents rather than a fixed call stack.

AutoGen is a strong fit for this pattern, since it is built around agents that converse to reach a goal. CrewAI can also be used here, and experimental setups like BabyAGI were built specifically to explore fully autonomous, goal-driven agent loops.

KEY POINTS

- You give the system a goal, not a procedure; the agents work out the steps and division of labor themselves.
- Multiple agents converse with each other, not only with the user.
- Best suited to open-ended problems where the right sequence of steps is not known in advance.
- AutoGen, CrewAI, and experimental tools like BabyAGI are built for this pattern.

Planner, coder, reviewer team

A coding assistant made of three agents that converse until the group is satisfied: a planner breaks the request into steps, a coder implements them, and a reviewer critiques the result and requests fixes. None of the three has a fixed script — the conversation itself determines what happens next.

```
# AutoGen-style autonomous multi-agent setup
planner = autogen.AssistantAgent(name="planner", system_message="Break the coding task
into steps.")
coder = autogen.AssistantAgent(name="coder", system_message="Write code for each step
the planner gives you.")
reviewer = autogen.AssistantAgent(name="reviewer", system_message="Review the coder's
output and request fixes.")
user_proxy = autogen.UserProxyAgent(name="user", human_input_mode="NEVER")

# Agents converse with each other until the group decides the goal is met
user_proxy.initiate_chat(
    planner,
    message="Build a function that deduplicates a list while preserving order."
)
```

04 Role-Based Multi-Agent Systems

6:23

Multiple agents still collaborate toward a shared goal, but each one operates inside a fixed role description instead of improvising its responsibilities.

Role-based systems are close cousins of autonomous multi-agent systems — both involve several agents working together. The difference is constraint: in a role-based system, each agent has a defined role, and it stays inside that role's boundaries even while communicating with the others. A content generation pipeline illustrates this well. A researcher agent goes out and gathers source material. A writer agent turns that material into an article. An editor agent reviews the draft and makes edits. They discuss the work with each other, but a researcher does not start writing prose, and an editor does not go fetch new sources — each stays inside the description it was given.

That constraint is the point. It trades some of the flexibility an autonomous system has for a division of labor that is easier to predict, test, and debug, because you always know which agent is responsible for which part of the output.

CrewAI is built directly around this role/task structure. AutoGen can also be used here if you add role constraints on top of its more general conversation model yourself. There are also niche frameworks built for one specific role-based task — ChatDev, for instance, targets software-development teams of agents specifically.

KEY POINTS

- Agents still talk to each other, but each has a role description that acts as a hard boundary.
- The difference from an autonomous multi-agent system is constraint: roles are fixed rather than emergent.
- A predictable division of labor makes debugging and quality control easier than fully open-ended collaboration.
- CrewAI is a strong fit; AutoGen works with added role structure; ChatDev is a niche framework built specifically for software-development role teams.

Researcher, writer, editor team

A content pipeline where a researcher gathers material, a writer drafts from it, and an editor refines the draft — each confined to its own role, defined with a CrewAI-style role, goal, and backstory.

```
researcher = Agent(role="Researcher", goal="Gather accurate source material on the
topic",
                    backstory="An analyst who only reports what sources actually say.")
writer = Agent(role="Writer", goal="Turn research into a clear draft article",
               backstory="A writer who never adds facts not present in the research.")
editor = Agent(role="Editor", goal="Tighten the draft and check it against the
research",
               backstory="An editor who refines wording, not the underlying claims.")

research_task = Task(description="Research {topic}", agent=researcher)
write_task = Task(description="Draft an article from the research", agent=writer)
edit_task = Task(description="Edit the draft for clarity and accuracy", agent=editor)

crew = Crew(agents=[researcher, writer, editor],
            tasks=[research_task, write_task, edit_task])
result = crew.kickoff(inputs={"topic": "agentic AI frameworks"})
```

05 Production Orchestration Systems

8:34

Once an agent moves out of experimentation and into a real system, it needs deep integration with APIs, databases, and business workflows, plus the reliability a prototype never had to have.

Production orchestration is less about a new coordination shape and more about a new set of requirements on top of whichever shape you're using. An AI operations agent is a representative example: it detects an alert, searches internal documentation for relevant runbooks, runs automation scripts to

remediate the issue, and sends a summary — all inside a real organization's infrastructure, where a wrong action has real consequences.

That context changes what the system needs from its framework. It is no longer enough for the logic to work in a demo; it has to integrate with live APIs and databases, handle failures without silently doing the wrong thing, and leave an audit trail so a human can see what happened and why.

Microsoft's Agent Framework — a merger of Semantic Kernel and AutoGen — is built directly for this purpose. LangGraph also fits here: because it structures workflows as a graph rather than a strict list, it can express the well-structured, multi-layered control flow that production systems need, in addition to the simpler linear workflows described earlier. That dual fit is why LangGraph shows up in both categories — the framework itself doesn't restrict you to one, the way you use it does.

KEY POINTS

- Production orchestration assumes the system already works conceptually; the job now is integration, reliability, and observability against real APIs and data stores.
- Microsoft's Agent Framework (Semantic Kernel + AutoGen) targets this pattern directly.
- LangGraph's graph structure lets it serve both well-structured linear workflows and more complex production orchestration.
- These systems wrap real side effects — running scripts, calling APIs, writing to databases — so error handling and audit trails matter more than in a prototype.

AI operations agent

An on-call agent that detects an alert, searches documentation for the matching runbook, runs a remediation script, and reports back — with explicit failure handling and logging, since a silent failure in production is worse than a slow one.

```
def handle_alert(alert):
    try:
        context = search_runbook_docs(alert.signature)
        plan = llm_decide_remediation(alert, context)
        result = run_automation_script(plan.script, dry_run=False)
        send_summary_to_oncall(alert, plan, result)
    except AutomationError as e:
        # production systems must fail safely, not silently
        escalate_to_human(alert, error=e)
        result = None
    log_run(alert, plan, result) # observability for the next incident
```

06 Rapid Prototyping Tools

9:57

When the goal is just to check whether an idea works at all, a drag-and-drop canvas trades architectural rigor for speed of validation.

Not every project starts by needing a well-architected system — sometimes the only question worth answering first is whether an idea works at all. Rapid prototyping tools are built for exactly that: a visual interface where you drag components onto a canvas, connect models and workflows, and test an idea without hand-writing the orchestration code underneath it.

The tradeoff is architectural depth for speed. These tools are not meant to be the final production system; they are meant to let you validate a concept quickly, then take what worked and rebuild it properly in a framework suited to whichever of the other four patterns the validated idea turns out to be.

LangFlow and Flowise are the examples here — both offer a graphical canvas for connecting models and workflow steps and quickly testing whether the resulting pipeline behaves the way you expect.

KEY POINTS

- The goal is a fast yes/no on whether an idea works, not a clean architecture.
- LangFlow and Flowise provide a visual canvas for connecting models and workflow steps without hand-writing orchestration code.
- Prototypes built this way are meant to be rebuilt properly once validated, not shipped to production as-is.

A visual flow, exported

Conceptually, dragging a prompt node into a model node into an output node on a LangFlow/Flowise-style canvas produces an underlying graph definition like this — the visual tool is generating and wiring this structure for you instead of you writing it by hand.

```
{
  "nodes": [
    {"id": "prompt-1", "type": "PromptTemplate", "data": {"template": "Summarize: {input}"}}},
    {"id": "llm-1", "type": "ChatModel", "data": {"model": "gpt-4o-mini"}},
    {"id": "output-1", "type": "TextOutput"}
  ],
  "edges": [
    {"source": "prompt-1", "target": "llm-1"},
    {"source": "llm-1", "target": "output-1"}
  ]
}
```

Glossary

Agentic AI framework

A toolkit that provides reusable coordination machinery — architectures, task/state management, monitoring, and communication protocols — for building AI systems that plan, act, and iterate rather than answer a single prompt.

Linear workflow

An agent pattern where steps execute in a fixed, predetermined sequence decided at design time rather than negotiated at runtime.

Autonomous multi-agent system

A pattern where multiple agents share a goal and work out the steps and division of labor between themselves, suited to open-ended problems.

Role-based multi-agent system

A multi-agent pattern where each agent operates inside a fixed role description rather than improvising its responsibilities, trading flexibility for predictability.

Production orchestration

A pattern focused on deep integration with live APIs, databases, and business systems, plus the reliability and observability a system needs once it leaves experimentation.

Rapid prototyping tool

A visual, low-code tool for quickly testing whether an agent idea works before committing to a full architecture.

LangChain

A framework for composing sequential chains of prompts, tools, and LLM calls, well suited to linear workflows.

LangGraph

A framework from the LangChain team that models agent workflows as a graph rather than a strict list, letting it handle both structured linear flows and more complex, stateful production orchestration.

CrewAI

A framework for building multi-agent teams, usable both for open-ended autonomous collaboration and for role-based teams with defined responsibilities.

AutoGen

A Microsoft framework for building conversable agents that exchange messages with each other to work toward a goal.

Agent Framework

Microsoft's merger of Semantic Kernel and AutoGen, aimed at production-grade agent orchestration with deep system integration.

Check yourself

Answer first, then open each one.

A team building a customer support bot needs the same three steps to run in the same order every time. Why would an autonomous multi-agent framework like AutoGen be a worse fit here than a linear-workflow framework like LangChain?

Autonomous multi-agent frameworks are built for agents to negotiate their own steps toward an open-ended goal. When the sequence is already fixed and known, that negotiation only adds unpredictability and debugging overhead without buying anything, whereas a linear-workflow framework simply executes the known order every time.

Two systems both use several collaborating agents — one is described as 'autonomous', the other as 'role-based'. Since both involve multi-agent negotiation, what is the actual structural difference between them?

In an autonomous system, the agents work out the plan and division of labor themselves at runtime. In a role-based system, each agent is constrained to a fixed role description and does not take on tasks outside that boundary, so the collaboration happens inside pre-set limits rather than emerging freely.

Why does moving an agent system into production typically demand different guarantees than the ones used to validate the idea in a tool like LangFlow?

Prototyping tools optimize for how fast you can test whether an idea works, not for handling live API failures, data integrity, or audit trails. Production orchestration adds the integration depth and reliability guarantees needed once the system touches real databases, APIs, and business processes where mistakes have real consequences.

LangGraph is described as fitting both the linear-workflow category and the production-orchestration category. How can the same framework belong to two different patterns?

LangGraph models workflows as a graph rather than a strict list, so it can express a simple sequential pipeline (a special case of a graph with no branches) as well as the more complex, stateful, multi-layered control flow production systems need. The category depends on how the graph is used, not a hard restriction built into the tool.

A single LLM call in a chatbot doesn't need an agentic framework. What actually changes once that same LLM is asked to analyze sales data, generate a report, and send it?

The system now has to plan multiple steps, call external tools such as a database and a calculation engine, hold and pass state between those steps, and possibly branch or iterate based on intermediate results — coordination a single prompt-response call never has to do. That coordination is exactly what a framework's predefined architectures and task management exist to handle.

Where to go next

- Pick one project idea and classify it into one of the five patterns before choosing a framework for it.
- Build a minimal LangChain LCEL chain for a two-step task to see a linear workflow first-hand.
- Compare AutoGen's conversable-agent model against CrewAI's role/task model on the same open-ended problem.
- Read LangGraph's documentation on graphs vs. chains to see why it spans both the linear-workflow and production-orchestration categories.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.